

ALEXANDRE TEIXEIRA

AI Engineer · GenAI & Production LLM Systems

alexandretei13@gmail.com ◇ +33 7 86 01 55 04 ◇ **LinkedIn** ◇ **GitHub** ◇ **Portfolio** ◇ Paris, Fr

Ingénieur IA 0→1 : je transforme des problématiques métier complexes en produits IA déployés et adoptés, du cadrage du besoin jusqu'au serving GPU en production. Mon approche : comprendre le terrain, simplifier la problématique, et construire de l'idée jusqu'à l'impact.

Formation

Université de Technologie de Compiègne (UTC) ***2022 – 2026***
Diplôme d'Ingénieur en Informatique (grade Master) Compiègne

- IA, optimisation, recherche opérationnelle, logique & résolution par recherche.
- Systèmes distribués, calcul parallèle, deep learning et apprentissage par renforcement (projets détaillés plus bas).

Shanghai University, UTSEUS (Département Franco-Chinois) ***sept. 2024 – janv. 2025***
Semestre d'échange en Intelligence Artificielle Shanghai

- Natural Language Processing, IA générative, analyse de données, data visualization.

Expérience professionnelle

Ingénieur IA, SNCF · Stage ***sept. 2025 – févr. 2026***
Plateforme IA multimodale de synthèse de réunions Paris

- **Problématique :** les RFI, visios et réunions en présentiel généraient des heures d'audio qu'aucune équipe n'avait le temps de dépouiller.
- **Solution :** conception complète et construction de **la plateforme 0→prod** : transcription, diarisation, synthèse, détection des slides clés et VLM pour injecter le contexte visuel dans les comptes rendus.
- Diarisation sur audio mono-canal et **fine-tuning d'un modèle speech-to-text** sur des voix SNCF réelles, sans golden dataset, pour des réunions internes confidentielles.
- Développement des briques de génération, d'organisation et de restitution des contenus ; application web React avec backend Python FastAPI.
- Industrialisation du déploiement via les outils CI/CD du groupe (Docker, Jenkins) sur Azure AI Services.
- Veille et prototypage sur le temps réel : API Realtime (WebRTC), modèles text-to-speech et speech-to-text.
- **Résultat : 90 % des audios traités en $<$ 5 min** ; plateforme adoptée par plusieurs équipes métier.
- **Référent technique** de l'équipe sur les orientations IA : **RFI menée sur 15 fournisseurs IA (Mistral AI, IBM, LinkUp, Vespa)** et accompagnement du déploiement interne de Mistral Code auprès des équipes d'ingénierie.

Environnement technique : Python, FastAPI, React, Azure AI Services, Azure OpenAI, Docker, Jenkins, VLM, STT / TTS, WebRTC.

Ingénieur IA, Mission Junior-Entreprise · Renault / Alpine ***mars 2025 – juin 2025***
Whiteboard IA pour industrialiser les AMDEC et infrastructure LLM sur GPU Compiègne

- **Problématique :** l'analyse de risques AMDEC était manuelle et dépendante des experts, ce qui limitait le nombre de processus couverts. Objectif fixé : **ramener la création d'une AMDEC de 6 mois à 2 mois**.
- **Solution :** conception d'un whiteboard collaboratif augmenté par IA : le LLM + RAG proposent risques, causes, effets et actions, les experts valident.
- **Architecture multi-agents** : un agent par sous-défaillance du système, avec liens entre les sous-solutions, et un agent pilote qui garantit des propositions distinctes et non redondantes.
- Composants LLM, bases vectorielles et pipelines de traitement documentaire pour structurer la connaissance métier et fiabiliser l'analyse des risques.
- Choix du dernier frontier model plutôt qu'une étude comparative préalable, qui aurait été périmée avant la fin du projet : un modèle plus performant est sorti deux mois après.
- **Infrastructure :** déploiement de modèles open-source sur un cluster **8× NVIDIA A100** ; serveur d'inférence **vLLM** (serving, batching, gestion mémoire GPU) et **load balancing** entre modèles et utilisateurs.
- Exposition des modèles via une **API compatible OpenAI** et suivi de l'infrastructure : disponibilité GPU, charge, mémoire et saturation des ressources.
- Travail en direct avec les interlocuteurs industriels de l'écosystème Renault-Nissan et Alpine.

Environnement technique : Python, LangChain, vLLM, Ollama, PostgreSQL, base vectorielle, Docker, NVIDIA A100.

Ingénieur IA, SNCF · Stage ***sept. 2023 – févr. 2024***
Assistant GenAI d'automatisation du support DevOps N1, devenu SNCF GPT Lyon

- **Problématique :** le groupe cherchait où le GenAI pouvait avoir un impact ; j'ai identifié le besoin sur le support DevOps N1, où les tickets ServiceNow répétitifs étaient traités manuellement.
- **Solution :** cadrage du cas d'usage et conception de l'assistant RAG **seul de 0 à 1**, de l'architecture au déploiement ; **MVP en 3 mois**.
- **Pipeline RAG :** parsing PDF / Markdown, nettoyage, découpage en chunks, extraction de métadonnées et embeddings ; stockage Azure Blob Storage, indexation vectorielle et sémantique avec Azure Cognitive Search, génération via Azure OpenAI.
- **Approche agentique :** agent ReAct et tool-calling pour aller chercher l'information à la bonne source (documentation interne, incidents historiques, Jira, SharePoint).
- **Intégrations :** API ServiceNow pour contextualiser les réponses, Microsoft Bot Framework / Azure Bot Service, Microsoft Graph API et webhooks Teams ; interface web React pour les équipes support.
- **Observabilité :** Langfuse pour la traçabilité des conversations, l'audit, les métriques LLM et le suivi des coûts ; expérimentations LangChain et LangGraph.
- **Industrialisation :** chaîne CI/CD GitLab, Jenkins, Ansible, Azure CLI et Vault ; images et artefacts sur JFrog Artifactory, Harbor et Docker Trusted Registry ; déploiement Kubernetes ; contrôles qualité et sécurité SonarQube et Checkmarx.
- **Résultat : 82 % des tickets N1 résolus automatiquement** ; produit porté par une équipe de 25 personnes, présenté sur BFM Business.
- Le produit a ensuite grandi à l'échelle du groupe pour **devenir SNCF GPT** (**3 000+ utilisateurs mensuels**), toujours en service aujourd'hui sur l'architecture et le mode agentique que j'ai conçus.

  Environnement technique : Python, Flask, Streamlit, React, Azure OpenAI, Azure Cognitive Search, Azure Blob Storage, Azure Web App, ServiceNow API, Microsoft Graph, Bot Framework, Langfuse, LangChain, LangGraph, Docker, Kubernetes, Terraform, Ansible, GitLab, Jenkins, Artifactory, Harbor, Vault, SonarQube, Checkmarx, Datadog, Jira.

Projets sélectionnés

Mikky, compagnon de bureau pour piloter ses agents de code ***sept. 2026 – auj.***
Projet personnel open source : suivre et superviser Claude Code et Codex sans fouiller dans ses terminaux. **GitHub**

- **Problématique :** avec plusieurs agents de code en parallèle, savoir où en est chaque tâche et répondre à leurs demandes oblige à jongler entre les terminaux.
- **Solution :** une île en haut de l'écran qui lance et suit les agents, affiche l'état et les étapes de chaque tâche, et remonte leurs demandes de permission avec leur contexte pour y répondre en un clic (Oui / Non / Toujours).
- **Moteur en Rust** (`mikkyd`) : lancement des agents via leurs adaptateurs ACP sous Windows ou WSL, suivi des sessions ouvertes ailleurs par lecture de leurs fichiers de session, hooks optionnels, métadonnées en SQLite.
- **App Flutter** : fenêtre transparente au-dessus des autres, forme dessinée par shader ; communication avec le moteur par WebSocket local, protégé par un jeton rangé dans le coffre Windows.
- 100 % local, sans compte ni télémétrie ; **0,013 % de CPU** île cachée ; tests sur de vraies sessions Claude / Codex rejouées et tests visuels des écrans.
- **Suite :** faire évoluer Mikky en « Discord pour agents », où les agents collaborent sur des projets, avec une gestion de projet type Jira nouvelle génération.

  Environnement technique : Rust, Flutter / Dart, WebSocket, SQLite, ACP, Claude Code, Codex, WSL.

Étude comparative CNN / Transformers pour l'imagerie médicale 3D, avec UCL ***févr. 2026 – auj.***
University College London : benchmark performance, complexité et contraintes d'inférence.

- Entraînement et évaluation de premiers modèles prototypes ; comparaison d'architectures CNN et Transformers sur images médicales 3D.
- Implémentation d'un CNN **from scratch** : convolution, pooling, fonctions d'activation, forward pass, backpropagation et boucle d'entraînement.

  Environnement technique : Python, PyTorch, CNN, Transformers.

Analyse de plans techniques et estimation de devis, usine de métallurgie ***mars – juin 2025***
Plateforme SaaS : computer vision + machine learning pour chiffrer le temps de fabrication des pièces.

- **Problématique :** le chiffrage des pièces reposait sur la lecture manuelle des plans techniques.
- Extraction automatique des données des plans par computer vision (YOLOv8, Transformers), puis modèle ML de prédiction du temps de fabrication et du devis.
- Chaîne de valeur complète : extraction, traitement, stockage et restitution via une interface web.
- Accompagnement d'utilisateurs non techniques pour constituer un historique de devis suffisant à l'entraînement du modèle.

  Environnement technique : Python, PyTorch, YOLOv8, Transformers, PostgreSQL, Next.js.

Plateforme LLM on-premise pour les étudiants (UTC) ***févr. – juil. 2025***
OpenWebUI + Ollama sur RTX 5090, hébergée sur les serveurs de l'UTC, ouverte à tous les étudiants.

– Déploiement et exploitation de la plateforme ; contrainte principale : faire tenir les modèles dans la VRAM du serveur.

Environnement technique : OpenWebUI, Ollama, NVIDIA RTX 5090.

AlphaZero from scratch, jeux Gopher & Dodo ***2024***

MCTS, self-play et deep RL ; réseau résiduel PyTorch avec têtes policy et value.

– Monte Carlo Tree Search couplé à un ResNet ; boucle d'entraînement par génération automatique de parties, policy loss, value loss et sauvegarde des modèles.

– Parallélisation pour accélérer l'entraînement et la recherche MCTS.

Environnement technique : Python, PyTorch.

Système distribué décentralisé avec vérification de données ***2025***

Réseau P2P en Go : nœuds capteurs, vérificateurs et utilisateurs.

– Réplicas locaux, horloges logiques, verrous distribués, file distribuée, peer discovery et propagation P2P des messages.

– Simulation de la vérification des données capteurs avec cohérence entre réplicas et prédiction utilisateur à partir des données locales.

Environnement technique : Go.

Analyse géospatiale du réseau hydrographique de Shanghai ***2024 – 2025***

Modélisation des rivières et canaux en graphe pour étudier connectivité et flux.

– Extraction et nettoyage des cours d'eau depuis OpenStreetMap ; enrichissement des géométries, segments, intersections et directions potentielles.

– Construction du graphe des waterways, features géospatiales pour modèles ML et premières approches de prédiction des flux ; stockage Parquet pour accélérer les lectures / écritures face au CSV.

Environnement technique : Python, Pandas, GeoPandas, OpenStreetMap, Parquet.

Moteur de recherche et visualisation, personnages de Game of Thrones ***2025***

Scraping, traitement de texte, NLP et génération de graphes de relations.

– Moteur de recherche sur un corpus scrapé et visualisation interactive des relations entre personnages.

Environnement technique : Python, NLP, scraping, graphes.

Expériences de recherche

Laboratoire des Technologies Innovantes (LTI), UPJV Amiens ***2022***

Gestion énergétique des bâtiments par deep learning.

– Modèles LSTM sur données capteurs (CO_2, bruit) pour détecter la présence de personnes et piloter la consommation d'énergie.

Laboratoire de l'UTC, calcul parallèle ***2025***

Calcul haute performance à petite échelle.

– Résolution d'équations de Poisson sur un mini supercalculateur pour évaluer le matériel avant passage sur de plus gros calculateurs (GENCI / Jean Zay).

Compétences techniques

IA / LLM Engineering: PyTorch, TensorFlow, Hugging Face, LangChain, LangGraph, Pydantic, RAG, embeddings, vector databases, VLM, STT / TTS, fine-tuning
Agents & LLMOps: agents ReAct, tool-calling, architectures multi-agents, orchestration d'agents de code (Claude Code, Codex, ACP), Langfuse (traçabilité, audit, coûts), évaluation de LLM
Serving & infra GPU: vLLM, Ollama, OpenWebUI, batching, gestion mémoire GPU, load balancing, API compatible OpenAI, NVIDIA A100 / RTX 5090
Machine Learning: CNNs, Transformers, ResNet, LSTM, YOLOv8, MCTS / RL, Scikit-learn, entraînement & évaluation de modèles
Cloud / DevOps: Azure AI Services, Azure OpenAI, Cognitive Search, Blob Storage, Docker, Kubernetes, Terraform, Ansible, Vault
CI/CD & qualité: GitLab CI/CD, Jenkins, JFrog Artifactory, Harbor, SonarQube, Checkmarx, Datadog
Intégrations: ServiceNow API, Microsoft Graph API, Bot Framework, webhooks Teams, REST APIs
Développement: Python, Rust, Go, C / C++, Dart / Flutter, SQL, React, Next.js, FastAPI, Flask, Streamlit, PostgreSQL, SQLite, WebSocket
Data & calcul scientifique: Pandas, NumPy, GeoPandas, Parquet, Matplotlib, R, Matlab, Scilab

Langues & engagements

Langues : Français (natif) · Anglais (professionnel) · Chinois (notions).
Responsable Partenariats, Imaginarium Festival : festival étudiant, sponsoring 600 000 EUR. ***2022 – 2023***
Founder Breakfast, Shanghai : échanges avec des fondateurs de startups sur l'innovation en IA. ***2024 – 2025***
Marchés de fruits et légumes : avec l'entreprise familiale depuis l'âge de 14 ans.
Centres d'intérêt : natation, course à pied, jeux vidéo, nouvelles technologies.